



UNIT 1

What Is Computer?

The word computer comes from the word “compute” which means, “to calculate”. A computer is an electronic device that can perform arithmetic operations at high speed and it can process data, pictures, sound, and graphics. It can solve highly complicated problems quickly and accurately.

A computer is also called a data processor because it can store, process, and retrieve data whenever desired

Computer receives input, stores, or processes the input as per user instructions and provides output in desired format.

A computer has two primary components

- **Hardware**-The term hardware refers to mechanical device that makes up computer. Computer hardware consists of interconnected electronic devices that we can use to control computer’s operation, input and output. Examples of hardware are CPU, keyboard, mouse, hard disk, etc.
- **Software**: A set of instructions that drives computer to do stipulated tasks is called a program. Software instructions are programmed in a computer language, translated into machine language, and executed by computer.

Software can be categorized into two types

(i) **System Software**-System software operates directly on hardware devices of computer. It provides a platform to run an application. It provides and supports user functionality. Examples of system software include operating systems such as Windows, Linux, Unix, etc.

(ii) **Application Software**-An application software is designed for benefit of users to perform one or more tasks. Examples of application software include Microsoft Word, Excel, PowerPoint, Oracle, etc.

Block diagram of Digital computer

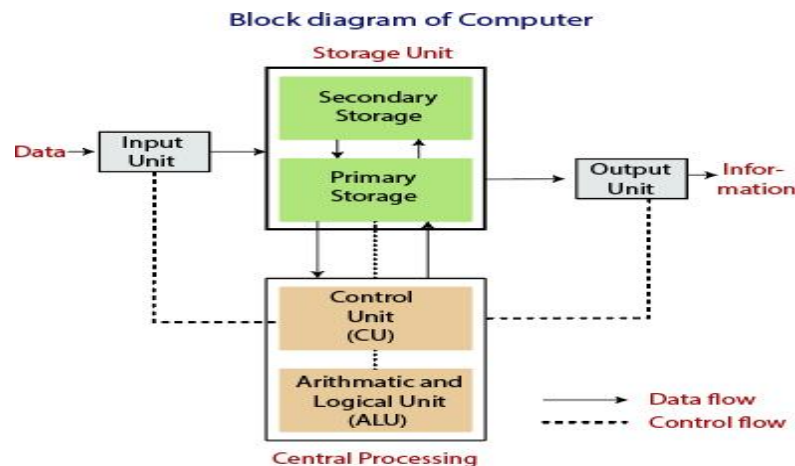


Fig: Block Diagram of the computer.

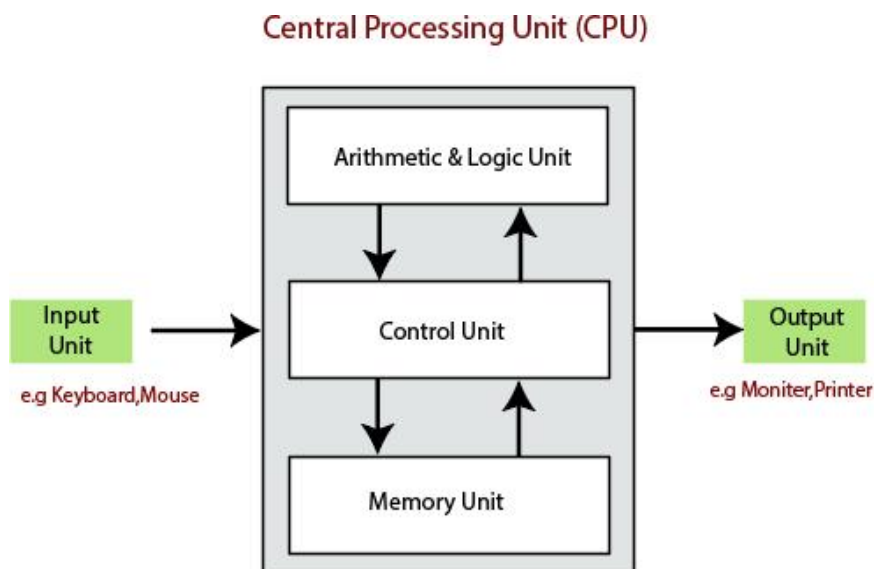
Block diagram of a computer gives you the pictorial representation of a computer that how it works inside. Or you can say that, in computers block diagram, you will see how computer works from feeding the data to getting the result.

Working of Digital Computer (Input-Processing-Output: IPO)

The data is entered through input devices such as the keyboard, mouse, etc. This set of instruction is processed by the CPU after getting the input by the user, and then the computer system produces the output. The computer can show the output with the help of output devices to the user, such as monitor, printer, etc.

Central Processing Unit (CPU)

The computer system is nothing without the Central processing Unit. So, it is also known as the brain of computer. The CPU is an electronic hardware device which can perform different types of operations such as arithmetic and logical operation.



The CPU contains two parts: the arithmetic logic unit and control unit. We have discussed briefly the arithmetic unit, logical unit, and control unit which are given below:

Control Unit

- The control unit (CU) controls all the activities or operations which are performed inside the computer system. It receives instructions or information directly from the main memory of the computer.
- When the control unit receives an instruction set or information, it converts the instruction set to control signals then; these signals are sent to the central processor for further processing. The control unit understands which operation to execute, accurately, and in which order.

Arithmetic and Logical Unit

- The arithmetic and logical unit is the combinational digital electronic circuit that can perform arithmetic operations on integer binary numbers. It presents the arithmetic and logical operation. The outputs of ALU will change asynchronously in response to the input. The basic arithmetic and bitwise logic functions are supported by ALU.

Storage Unit

The information or set of guidelines are stored in the storage unit of the computer system. The storage unit provides the space to store the data or instruction of processed data. The information or data is saved or hold in computer memory or storage device. The data storage is the core function and fundamental of the computer components.

There are mainly two types of storage Unit-

- (i) Primary Storage
- (ii) Secondary Storage

- (i) **Primary memory** is the main memory of computer system. The instructions that must be currently executed is copied to the primary memory because CPU can directly access the data from primary memory. Accessing data from primary memory is faster as it is an internal memory and processor accesses data from primary memory using data bus.
- (ii) **Secondary storage** is also called as permanent storage unit as when the data store in this memory are stored permanently user can recall the data whenever they need. The data remains in this memory even when there is a power failure or shutdown, they are much cheaper than primary memory. The Example of Secondary Storage are Hard disk and Zip drives.

Input Devices

The user provides the set of instruction or information to the computer system with the help of input devices such as the keyboard, mouse, scanner, etc.

Output Devices

The output devices produce or generate the desired result according to our input, such as a printer, monitor, etc. These devices convert the data into a human-readable form from binary code.

Computer Memory:

Memory is storage part of computer. It stores data, information, programs during processing in computer.

It stores data either temporarily or permanent basis. Memory has important roles in saving and retrieve data.

Types of memory:

Mainly computer has two types of memory

1. Primary Memory/ Volatile Memory
2. Secondary Memory/ Non-Volatile Memory

RAM (Random Access Memory):

Random Access Memory (RAM) is a type of computer memory that is used to temporarily store data that the computer is currently using or processing. RAM is volatile memory, which means that the data stored in it is lost when the power is turned off. RAM is typically used to store the operating system, application programs, and data that the computer is currently using.

Types of Random Access Memory (RAM)

- **Static RAM (SRAM):** SRAM stands for Static Random Access Memory. It is a type of semiconductor which is widely used in computing devices and microprocessors. Retain stored information if the power supply is on. SRAM of higher cost and consumes more power. They have higher speed than DRAM.
- **Dynamic RAM (DRAM):** It stored information in a very short time (a few millisecond) even though the power supply is ON. The DRAM is cheaper and moderate speed and, they consume less power. It is made of Capacitors and has smaller data life span than Static RAM.

ROM(Read Only Memory):

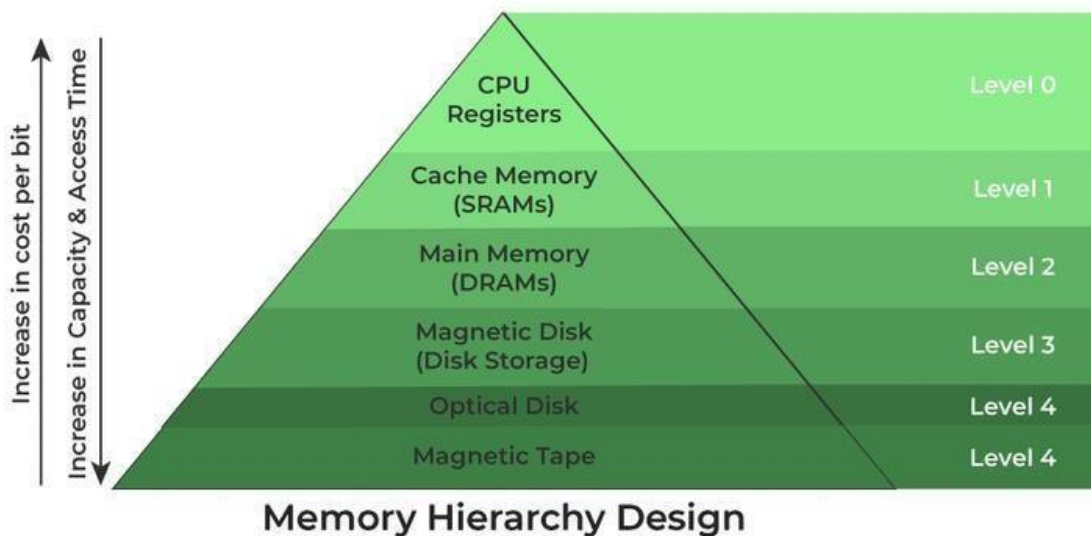
Read Only Memory (ROM) is a type of computer memory that is used to permanently store data that does not need to be modified. ROM is non-volatile memory, which means that the data stored in it is retained even when the power is turned off. ROM is typically used to store the computer's BIOS (basic input/output system), which contains the instructions for booting the computer, as well as firmware for other hardware devices.

Types of Read-Only Memory (ROM)

- **PROM (Programmable read-only memory):** It can be programmed by the user. Once programmed, the data and instructions in it cannot be changed.
- **EPROM (Erasable Programmable read-only memory):** It can be reprogrammed. To erase data from it, expose it to ultraviolet light. To reprogram it, erase all the previous data.
- **EEPROM (Electrically erasable programmable read-only memory):** The data can be erased by applying an electric field, with no need for ultraviolet light. We can erase only portions of the chip.

Memory Hierarchy

Memory Hierarchy is an enhancement to organize the memory such that it can minimize the access time. The Memory Hierarchy was developed based on a program behaviour known as locality of references.



This Memory Hierarchy Design is divided into 2 main types:

- **External Memory or Secondary Memory:** Comprising of Magnetic Disk, Optical Disk, and Magnetic Tape i.e. peripheral storage devices which are accessible by the processor via an I/O Module.
 - **Internal Memory or Primary Memory:** Comprising of Main Memory, Cache Memory & CPU registers. This is directly accessible by the processor.
- i. **Registers** are small, high-speed memory units located in the CPU. They are used to store the most frequently used data and instructions. Registers have the fastest access time and the smallest storage capacity, typically ranging from 16 to 64 bits.
 - ii. **Cache memory** is a small, fast memory unit located close to the CPU. It stores frequently used data and instructions that have been recently accessed from the main memory. Cache memory is designed to minimize the time it takes to access data by providing the CPU with quick access to frequently used data.
 - iii. **Main memory**, also known as RAM (Random Access Memory), is the primary memory of a computer system. It has a larger storage capacity than cache memory, but it is slower. Main memory is used to store data and instructions that are currently in use by the CPU.
 - iv. **Secondary storage**, such as hard disk drives (HDD) and solid-state drives (SSD), is a non-volatile memory unit that has a larger storage capacity than main memory. It is used to store data and instructions that are not currently in use by the CPU. Secondary storage has the slowest access time and is typically the least expensive type of memory in the memory hierarchy.
 - v. **Magnetic Disks**, are simply circular plates that are fabricated with either a metal or a plastic or a magnetized material. The Magnetic disks work at a high speed inside the

computer and these are frequently used.

- vi. **Magnetic Tape**, is simply a magnetic recording device that is covered with a plastic film. It is generally used for the backup of data. In the case of a magnetic tape, the access time for a computer is a little slower and therefore, it requires some amount of time for accessing the strip.

Characteristics of Memory Hierarchy

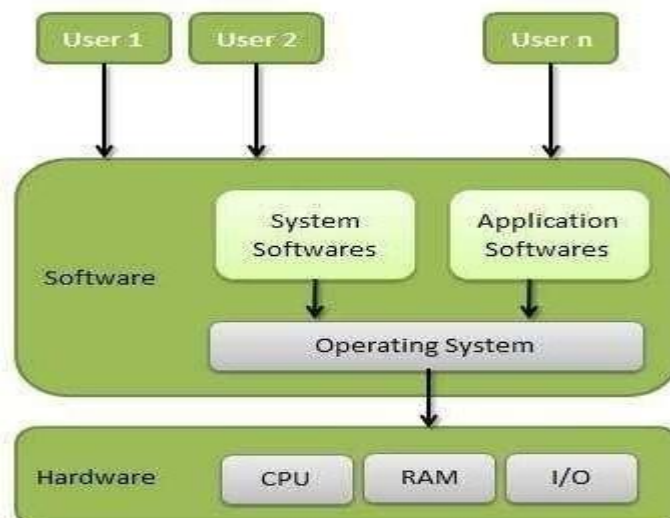
- **Capacity:** It is the global volume of information the memory can store. As we move from top to bottom in the Hierarchy, the capacity increases.
- **Access Time:** It is the time interval between the read/write request and the availability of the data. As we move from top to bottom in the Hierarchy, the access time increases.
- **Performance:** Earlier when the computer system was designed without a Memory Hierarchy design, the speed gap increased between the CPU registers and Main Memory due to a large difference in access time. This results in lower performance of the system and thus, enhancement was required. This enhancement was made in the form of Memory Hierarchy Design because of which the performance of the system increases.
- **Cost Per Bit:** As we move from bottom to top in the Hierarchy, the cost per bit increases i.e. Internal Memory is costlier than External Memory.

Operating System:

An Operating system (OS) is a system software which acts as an interface between the end user and computer hardware. Every computer must have at least one OS to run other programs. An applications like Chrome, MS Word, Games, etc needs some environment in which it will run and perform its task.

Operating System as a Resource Manager

Internally an Operating System acts as a resource manager of the computer system such as processor, memory, files, and I/O device.



Functions of Operating system

- **Process management:-** Process management helps OS to create and delete processes. It also provides mechanisms for synchronization and communication among processes.
- **Memory management:-** Memory management module performs the task of allocation and de-allocation of memory space to programs in need of this resources.
- **File management:-** It manages all the file-related activities such as organization storage, retrieval, naming, sharing, and protection of files.
- **Device Management:** Device management keeps tracks of all devices. This module also responsible for this task is known as the I/O controller. It also performs the task of allocation and de-allocation of the devices.
- **I/O System Management:** One of the main objects of any OS is to hide the peculiarities of that hardware devices from the user.
- **Security:-** Security module protects the data and information of a computer system against malware threat and authorized access.
- **Networking:** A distributed system is a group of processors which do not share memory, hardware devices, or a clock. The processors communicate with one another through the network.
- **Communication management:** Coordination and assignment of compilers, interpreters, and another software resource of the various users of the computer systems.

Computer Language

A **computer language** is a group of instructions that are used to create computer programs. A **programming language** is a system of notation for writing computer programs. A programming language is a **computer language** that is used by **programmers (developers)** to communicate with computers. It is a set of instructions written in any specific language (C, C++, Java, Python) to perform a specific task.

Types of programming language

Computer Language is divided into 2 types.

- **Low-level programming language**

Low-level language is **machine-dependent (0s and 1s)** programming language. The processor runs low- level programs directly without the need of a compiler or interpreter, so the programs written in low-level language can be run very fast.

Low-level language is further divided into two parts -

i. Machine Language

Machine language is a type of low-level programming language. It is also called as

machine code or object code. Machine language is easier to read because it is normally displayed in binary or hexadecimal form (base 16) form. It does not require a translator to convert the programs because computers directly understand the machine language programs.

The advantage of machine language is that it helps the programmer to execute the programs faster than the high-level programming language.

ii. Assembly Language

Assembly language (ASM) is also a type of low-level programming language that is designed for specific processors. It represents the set of instructions in a **symbolic and human-understandable form**. It uses an assembler to convert the assembly language to machine language.

The advantage of assembly language is that it requires less memory and less execution time to execute a program.

Advantage of LLL:-

1. As it is directly understandable by the machine, processing speed is high.
2. No translator is required.
3. It occupies less memory space.

Disadvantages of LLL:-

1. LLL is difficult to read and write.
2. Difficult to debug.
3. It is not portable.
4. It is platform dependent.

● High-level programming language

High-level programming language (HLL) is designed for **developing user-friendly software programs and websites**. This programming language requires a compiler or interpreter to translate the program into machine language (execute the program).

The main advantage of a high-level language is that it is **easy to read, write, and maintain**.

High-level programming language includes **Python, Java, JavaScript, PHP, C#, C++, Objective C, Cobol, Perl, Pascal, LISP, FORTRAN, and Swift programming language**.

Advantages of HLL

1. HLL is easy to read and write.
2. HLL is easy to debug.
3. HLL is machine independent.
4. HLL is platform independent.

Disadvantages of HLL

1. Processing speed is slow, as it needs translators.
2. It occupies more memory space.

Translators:

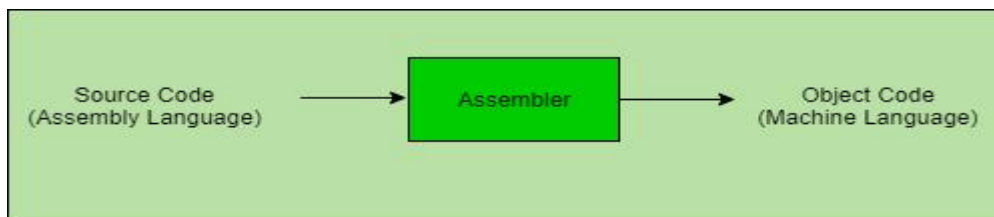
Computer programs are generally written in high-level languages (like C++, Python, and Java). A language processor, or language translator, is a computer program that convert source code from one programming language to another language or human readable language. They also find errors during translation.

There are three types of translators.

1. **Assembler**
2. **Compiler**
3. **Interpreter**

1. Assembler:

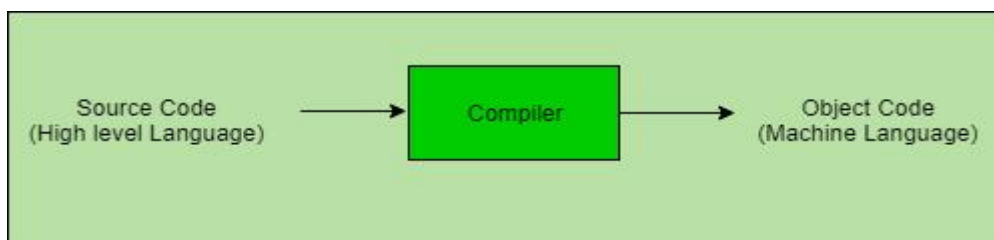
An assembler converts programs written in assembly language into machine code. It is also referred to assembler as assembler language by some users. The source program has assembly language instructions, which is an input of the assembler. The assemble translates this source code into a code that is understandable by the computer, called object code or machine code.



2. Compiler:

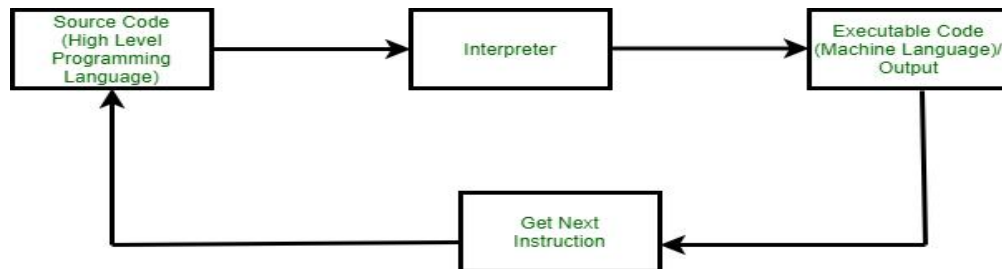
The language processor that reads the complete source program written in high-level language in one go and translates it into an equivalent program in machine language is called a Compiler. Example: C, C++, C#.

In a compiler, the source code is translated to object code successfully if it is free of errors. The compiler specifies the errors at the end of the compilation with line numbers when there are any errors in the source code. The errors must be removed before the compiler can successfully recompile the source code again the object program can be executed number of times without translating it again.



3. Interpreter:

The programs written with the help of using one of the many high-level programming languages are directly executed by an interpreter without previously converting them to an object code or machine code, which is done line by line or statement by statement. When the interpreter is translating the source code, it displays an error message if there is an error in the statement and terminates this statement from translating process. When the interpreter removed errors on the first line, then it moves on to the next line.

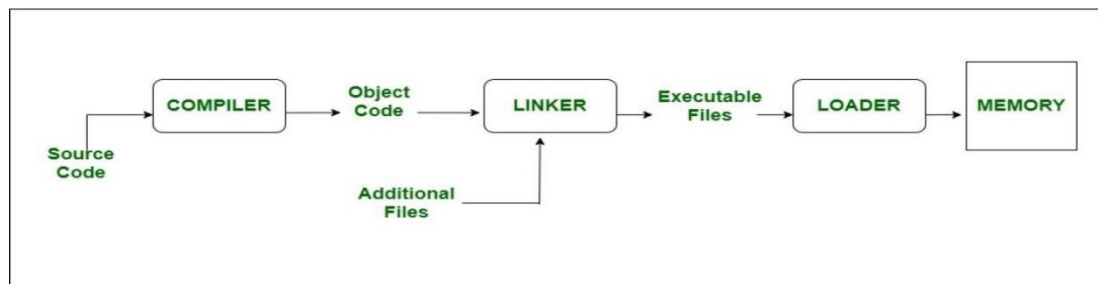


Linker:

A linker is special program that combines the object files, generated by compiler/assembler and other pieces of code to originate an executable file has .exe extension. In the object file, linker searches and append all libraries needed for execution of file. It regulates the memory space that will hold the code from each module. It also merges two or more separate object programs and establishes link among them.

Loader:

It is special program that takes input of executable files from linker, loads it to main memory, and prepares this code for execution by computer. Loader allocates memory space to program.



Idea of Program / Algorithm

Program:

A **program** is a set of instructions written in a programming language (like C, Python, Java) to solve a problem. A program is the **practical implementation** of a solution that can be executed by a computer.

Algorithm :

- An **algorithm** is a **step-by-step procedure** to solve a problem.
- It is **language-independent** (not written in C, Java, etc., but in simple steps).

Properties of a good algorithm:

- **Finiteness** – must end after finite steps.
- **Definiteness** – each step must be clear.
- **Input** – takes zero or more inputs.
- **Output** – produces at least one output.
- **Effectiveness** – steps must be basic and doable.

Example Algorithm: To add two numbers

Start

1. Read two numbers A and B
2. Calculate Sum = A + B
3. Print Sum

Stop

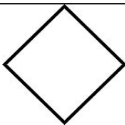
Representation of Algorithms:

Algorithms can be represented in two common ways:

1. Flowchart:

A diagrammatic representation of an algorithm.

Uses standard symbols:

Symbol Name	Symbol Representation
Start/ End	
Process	
Decision	
Input/Output	
Flow Arrow	
On-page connector/reference	

2. Pseudo-code

Pseudocode is a compact and informal high-level description of an algorithm that uses the structural conventions of a programming language. It is an artificial and informal language that helps programmer to develop program. Pseudocode is text- based detail design tool. Pseudocode is an intermediary between an algorithm and implementation. Pseudocode is an outline of program that can be easily converted into programming statements.

Example: Write Pseudocode to add two numbers

START

INPUT number1

INPUT number2

sum $\leftarrow$ number1 + number2

PRINT: sum

END

Writing and Executing the First C++ Program

1. Understanding what C++ is

C++ is a programming language used to write programs (instructions for the computer). A C++ program is written in source code (plain text) and then converted into machine code (binary) using a compiler so the computer can run it.

2. Steps to Write and Execute a C++ Program

A) Install a C++ Compiler

- For Windows: Install MinGW or use Code::Blocks / Dev C++.
- For Linux: Usually already has g++ compiler.
- For Mac: Install Xcode or use Homebrew (brew install gcc).

B) Write Your First Program

Open any editor (Notepad, VS Code, Code::Blocks, etc.) and type this code:

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello, World!";
    return 0;
}
```

Explanation of the code:

- #include <iostream> → Tells the program to use input/output commands.
 - using namespace std; → Allows us to use cout without writing std::cout.
 - int main() → Starting point of every C++ program.
 - cout << "Hello, World!"; → Prints text on the screen.
 - return 0; → Indicates program ended successfully.
-

C) Save the File

Save the file with .cpp extension. Example: first.cpp

D) Compile the Program

Open Command Prompt (Windows) or Terminal (Linux/Mac). Go to the folder where you saved the file (e.g., cd Desktop). Type the compile command:
g++ first.cpp -o first

Here:

g++ → **Compiler command**

first.cpp → **Source file**

-o first → **Output file (executable)**

E) Run the Program

After compilation, run it:

- On Windows: first.exe
- On Linux/Mac: ./first

Output:
Hello, World!

3. Common Mistakes Beginners Make

- Forgetting ';' at the end of a statement.
- Saving file without .cpp extension.
- Running program without compiling first.

4. Next Steps After First Program

Try changing the text in cout.
Try printing numbers.
Try taking input using cin.

```
#include <iostream>
using namespace std;

int main() {
    int a, b;
    cout << "Enter two numbers: ";
    cin >> a >> b;
    cout << "Sum = " << a + b;
    return 0;
}
```

Syntax and Logical Errors in Compilation

When writing C++ programs, we can make different kinds of mistakes. Two common ones are Syntax Errors and Logical Errors.

1. Syntax Errors

- Meaning: Syntax errors happen when you break the rules of the C++ language grammar.
- The compiler cannot understand the code and will stop compiling.
- These are like spelling or grammar mistakes in English.

Example of Syntax Error:

```
#include <iostream>
using namespace std;

int main() {
    int num
    cout << "Enter a number: "; // Missing semicolon above
    cin >> num;
    cout << "You entered: " << num;
    return 0;
}
```

Error: Missing semicolon after 'int num'.
The compiler will give an error message like:
error: expected ';' before 'cout'

Corrected Code:

```
int num;
```

2. Logical Errors

- Meaning: Logical errors happen when the program runs but gives the wrong result because the logic (thinking) of the code is wrong.
- The compiler will not detect this — it's up to the programmer to find it.
- These are like making a mistake in math calculation even though grammar is correct.

Example of Logical Error:

```
#include <iostream>
using namespace std;

int main() {
    int a = 10, b = 5;
    int sum = a - b; // Wrong logic: used '-' instead of '+'
    cout << "The sum is: " << sum;
    return 0;
}
```

Output will be: The sum is: 5
But we wanted addition, so the correct output should be 15.

Corrected Code:

```
int sum = a + b;
```

Key Difference Between Syntax and Logical Errors

Feature	Syntax Error	Logical Error	Example
When it occurs	During compilation	During execution (runtime output wrong)	-
Detected by	Compiler	Programmer only (compiler allows it)	-
Program Execution Example	Program won't compile/run Missing semicolon, wrong keyword	Program runs but gives wrong result Wrong formula, wrong operator	- -

Object Code and Executable Code in C++

1. Source Code

The program that a programmer writes in a high-level language like C++ is called Source Code.

Example:

```
#include <iostream>
using namespace std;
int main() {
    cout << "Hello World";
    return 0;
}
```

2. Compilation Process

The C++ compiler translates source code into a lower-level form that the computer can understand. This process involves several stages:

- **Preprocessing** – Handles #include, #define, and macros.
- **Compilation** – Converts the preprocessed source code into Assembly Code.
- **Assembly** – Translates the assembly code into Object Code (.obj or .o file).
- **Linking** – Combines object code with required libraries to create the final Executable Code (.exe file).

3. Object Code

Definition: Object code is the machine-level representation of your program, generated after compilation but before linking.

File Extension: .o (Linux) or .obj (Windows).

Features:

- Not directly human-readable.
- Contains machine instructions specific to the processor.
- Incomplete program (needs linking with libraries).

Example: When you compile program.cpp, the compiler produces an intermediate file program.obj.

4. Executable Code

Definition: Executable code is the final machine code that can be run directly by the computer's operating system.

File Extension: .exe (Windows) or no extension (Linux).

Features:

- Ready-to-run program.
- Generated after linker combines object code + standard libraries (like iostream).
- Fully translated into binary instructions for the CPU.

Example: program.exe is the executable file that prints "Hello World" when run.

Difference between Object Code and Executable Code

Aspect	Description
Generation	Object Code: Produced after compilation Executable Code: Produced after linking
File Extension	Object Code: .obj / .o Executable Code: .exe (Windows) or none (Linux)
Completeness	Object Code: Incomplete program Executable Code: Complete, runnable program
Readability	Object Code: Not human-readable Executable Code: Not human-readable
Purpose	Object Code: Intermediate step before execution Executable Code: Final output, ready to run

Summary

Source Code → Compiler → Object Code → Linker → Executable Code

- Object Code is an intermediate product.
- Executable Code is the final product that can be executed by the operating system.

Structure of a C++ Program

A C++ program is built using several building blocks. Understanding its structure is very important for writing correct and efficient programs. The general structure of a C++ program is as follows:

1. Preprocessor Directives (Header Files): These are instructions to the compiler to include libraries before compilation.
2. Namespace Declaration: Allows access to names (like `cout`, `cin`) without prefixing them with `std::`.
3. `main()` Function: The entry point of the program, where execution begins.
4. Body of the Program: Contains statements, variables, functions, and logic.
5. Comments: Non-executable statements used to explain the program.

Example of a simple C++ program:

```
#include <iostream>
using namespace std;

int main() {
    // Display message
    cout << "Hello, World!";
    return 0;
}
```

The `main()` Function

The `main()` function is the starting point of any C++ program. The compiler always looks for `main()` first. The body of the `main()` function contains the instructions that will be executed.

Syntax:

```
int main() {
    // statements
    return 0;
}
```

- `int`: Indicates that the function returns an integer value.
- `return 0`: Signifies successful execution of the program.
- We can also use `return 1` or other values to indicate errors.

Header Files

Header files provide pre-written code (functions, classes, objects) that we can reuse in our program. Instead of writing code from scratch, we include them using the `#include` directive.

Common Header Files:

- <iostream>: Provides input and output functions (cin, cout).
- <cmath>: Provides mathematical functions like sqrt(), pow().
- <string>: Provides functions to handle strings.
- <vector>: Provides dynamic array functionality.
- <iomanip>: Used for formatting output.

Example:

```
#include <iostream>
#include <cmath>
int main() {
    double num = 16;
    cout << sqrt(num); // prints 4
    return 0;
}
```

Standard Input and Output

C++ uses streams for input and output operations.

- cout (character output stream): Used to display data on the screen.
- cin (character input stream): Used to take data from the keyboard.
- cerr: Used for error messages.
- clog: Used for logging information.

Example of cin and cout:

```
int age;
cout << "Enter your age: ";
cin >> age;
cout << "Your age is: " << age;
```

Comments in C++

Comments are non-executable lines of code that help programmers understand the logic of a program. They are ignored by the compiler but very useful for documentation.

Types of Comments:

1. Single-line Comments: Begin with //

Example: // This is a single-line comment

2. Multi-line Comments: Enclosed within /* */

Example:

```
/*
    This is a
    multi-line comment
*/
```

Flow of Execution in a C++ Program

The flow of execution in a C++ program follows these steps:

1. Preprocessor includes header files.
2. Compilation of the program.
3. Linking with necessary libraries.
4. Execution starts with main().
5. Program statements are executed sequentially.
6. Program terminates when return statement is reached.

Flow Representation:

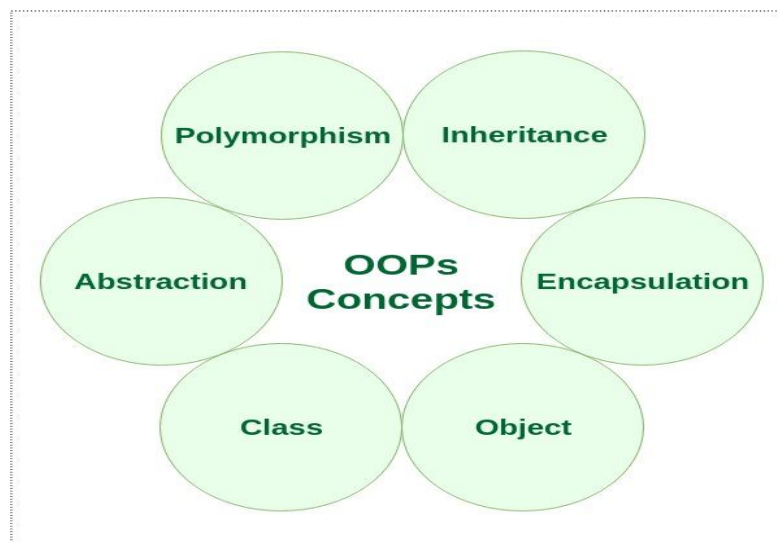
Start → Preprocessor Directives → Compilation → main() → Program Execution → End

Introduction to Object-Oriented Programming

Object Oriented Programming (OOP) is a style of programming that uses objects to model real-world things like data and behavior. It focuses on key ideas like inheritance, encapsulation, and polymorphism. The main goal of OOP is to group data and the functions that work on it together, so that the data is protected and can only be changed in controlled ways.

Characteristics of an Object-Oriented Programming Language

There are some basic concepts that act as the building blocks of OOPs i.e.



Class

In C++, the basic building block of Object-Oriented Programming is the class. A class is a user-defined type that acts like a blueprint to create objects that share similar properties (data) and behaviors (functions).

For example, you can define an Animal class with properties like name, age, and species, and behaviors like eat(), sleep(), and makeSound().

A class is defined using the class keyword in C++.

```
class Animal
{
    public:
        string species;
        int age;
        int name;

        // Member functions
        void eat()
        {
            // eat something
        }

        void sleep()
        {
            // sleep for few hrs
        }

        void makeSound ()
        {
            // make sound
        }

};
```

Here, public is the access specifier that specify what functions are available to others.

Object

An object is a real, usable instance of a class that has specific properties and behaviors. In C++, an object is created from a class.

For example, Animal is just an idea or blueprint, but a cat is a real object based on that class. So, classes are concepts, and objects are the actual things created from those concepts.

When you define a class, no memory is used. But when you create an object from that class, memory is allocated for it. You can create an object like this:

```

#include <iostream>
using namespace std;

class Animal
{
public:
    string species;
    int age;
    int name;

    // Member functions
    void eat()
    {
        // eat something
    }
    void sleep()
    {
        // sleep for few hrs
    }
    void makeSound()
    {
        // make sound;
    }
};

int main()
{
    Animal cat;
}

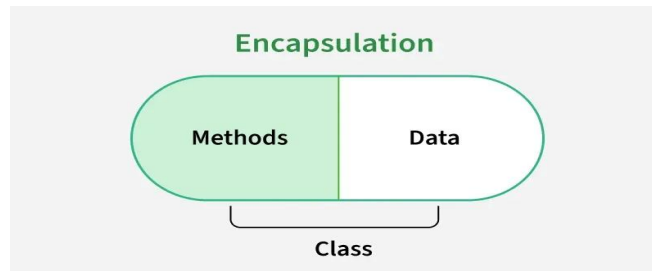
```

Objects take up space in memory and have an associated address like a record in pascal or structure or union. When a program is executed, the objects interact by sending messages to one another. Each object contains data and code to manipulate the data. Objects can interact without having to know details of each other's data or code, it is sufficient to know the type of message accepted and the type of response returned by the objects.

Encapsulation

In simple terms, encapsulation is defined as wrapping up data and information under a single unit. In Object-Oriented Programming, encapsulation is defined as binding together the data and the functions that manipulate them together in a class.

Consider an example of the Animal class, the data members species, age and name are encapsulated with the member functions like eat(), sleep, etc. They can be protected by the access specifier which hides the data of the class from outside.



Abstraction

Abstraction is one of the most essential and important features of object-oriented programming in C++. Abstraction means displaying only essential information and ignoring the other details. Data abstraction refers to providing only essential information about the data to the outside world, hiding the background details or implementation. In our case, when we call the `makeSound()` method, we don't need to know how the sound is produced internally, only that the method makes the animal sound.

Polymorphism

The word polymorphism means having many forms. In simple words, we can define polymorphism as the ability of an entity to behave different in different scenarios. person at the same time can have different characteristics.

For example, the same `makeSound()` method, the output will vary depending on the type of animal. So, this is an example of polymorphism where the `makeSound()` method behaves differently depending on the Animal type (Dog or Cat).

Inheritance

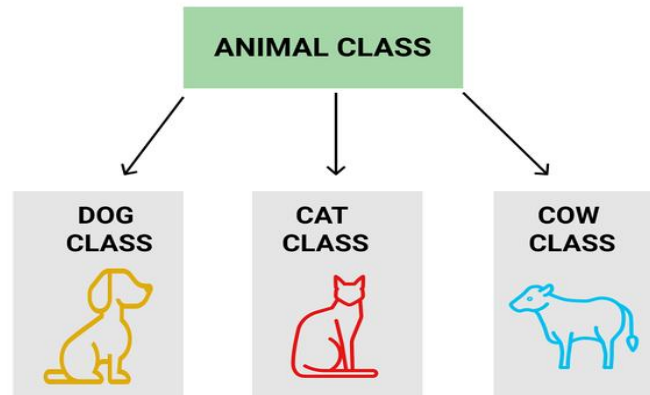
The capability of a class to derive properties and characteristics from another class is called Inheritance. Inheritance is one of the most important features of Object-Oriented Programming.

Sub Class: The class that inherits properties from another class is called Sub class or Derived Class.

Super Class: The class whose properties are inherited by a sub-class is called Base Class or Superclass.

Inheritance supports the concept of “reusability”, i.e. when we want to create a new class and there is already a class that includes some of the code that we want, we can derive our new class from the existing class. By doing this, we are reusing the fields and methods of the existing class.

Example: Dog, Cat, Cow can be Derived Class of Animal Base Class.



Inheritance in C++

Advantage of OOPs

- Here are key advantages of Object-Oriented Programming (OOP) over Procedure-Oriented Programming (POP):
- **Modularity and Reusability:** OOP promotes modularity through classes and objects, allowing for code reusability.
- **Data Encapsulation:** OOP encapsulates data within objects, enhancing data security and integrity.
- **Inheritance:** OOP supports inheritance, reducing redundancy by reusing existing code.
- **Polymorphism:** OOP allows polymorphism, enabling flexible and dynamic code through method overriding.
- **Abstraction:** OOP enables abstraction, hiding complex details and exposing only essential features

Basic Structure of Class in C++ and Creating Objects

In C++, a class is a user-defined data type that acts as a blueprint for creating objects. It combines data members (variables) and member functions (methods) into a single unit, representing real-world entities in programming.

1. Basic Structure of a Class in C++

The general structure of a class in C++ is as follows:

```

class ClassName
{
    access_specifier:
        // data members (variables)
        // member functions (methods)
};

```

Explanation of components:

- **class:** Keyword used to declare a class.
- **ClassName:** The name of the class (user-defined).
- **access_specifier:** Defines the visibility of class members. Common specifiers are:
 - **public:** Accessible from outside the class.
 - **private:** Accessible only within the class (default).
 - **protected:** Accessible within the class and derived classes.
- **Data Members:** Variables that hold the data of the object.
- **Member Functions:** Functions that define the behavior of the object.
- **Semicolon (;):** A class declaration must end with a semicolon.

Example:

```

class Car
{
    public:
        string brand;
        int speed;

        void display()
        {
            cout << "Brand: " << brand << endl;
            cout << "Speed: " << speed << " km/h" << endl;
        }
};

```

2. Creating Objects in C++

An object is an instance of a class. Once a class is defined, we can create multiple objects that use the same structure but can store different values.

Syntax:

ClassName objectName;

Example:

```
int main()
{
    Car car1; // creating first object
    Car car2; // creating second object

    car1.brand = "Toyota";
    car1.speed = 120;

    car2.brand = "BMW";
    car2.speed = 150;

    car1.display();
    car2.display();

    return 0;
}
```

Explanation:

- car1 and car2 are objects of the Car class.
- Each object has its own copy of data members (brand and speed).
- The member function display() is used to show the details of each object.

Conclusion

In C++, classes define the blueprint for objects, and objects are the instances that bring these blueprints to life. Understanding the basic structure of a class and how to create objects is the foundation of Object-Oriented Programming (OOP).